JOINT BENCHMARK INITIATIVE — EVOCHIP.AI & SIDEPATH

AltiCoreAI Benchmark Report

CPU Inference Throughput Evaluation Across Seven Public Datasets — Version 6.0

7	13–21×	17–28×	575 M	0
PUBLIC DATASETS	FASTER — WORKSTATION	FASTER — SERVER	PEAK INF/SEC	DATASETS ALTICORE SLOWER

DOCUMENT	SCOPE	PLATFORMS	ISSUED BY
Version 6.0	CPU-Only Inference	Workstation & Server x86 (AVX2)	EvoChip.ai / SidePath

TABLE OF CONTENTS

SECTION 01

EvoChip.ai

EvoChip.ai is a compute-architecture company focused on redefining inference efficiency across software, edge, and hardware-integrated systems. Rather than incrementally optimizing existing neural-network execution stacks, EvoChip develops **mathematically distinct inference architectures** designed from first principles to maximize computational efficiency, speed, simplicity and energy savings on general-purpose, embedded, MCU and programmable logic.

AltiCore, EvoChip.ai's new mathematical framework, is the patented IP, based on multiple deployment layers. Across these layers, EvoChip applies a consistent architectural philosophy: inference should be **efficient and**

economical, regardless of where it runs with whatever resources are practical within constraints. This approach aligns particularly well with applications where inference cost, predictability, and system-level integration matter more than maximum model expressiveness.

SECTION 02

SidePath

SidePath is a premier IT solutions provider and consultancy dedicated to modernizing technology infrastructures. Specializing in cloud computing, data protection, and virtualization, the company helps organizations optimize their data centers and streamline operations.

Through strategic partnerships with industry leaders, SidePath delivers tailored, scalable solutions designed to meet the performance and security demands of today's digital landscape.

SECTION 03

AltiCore Technology

AltiCore is not "another neural network implementation." It is a new mathematical approach built to **lean into what computers do natively — fast logical operations and hardware-native primitives — while minimizing dependence on heavy arithmetic.**

In the current state of development, the practical result is an approach optimized for **compact models** (low parameter counts and low variable counts). That matters for two real-world reasons: it can make inference **dramatically faster on CPU**, and it can make AI **deployable on constrained environments where conventional approaches often cannot run at all**. This performance advantage translates directly into **lower cost per decision**, **higher inference capacity per hardware**, and **simpler, more scalable deployments**.

AltiCoreAI is the umbrella brand for all AI use-case products and technologies built on AltiCore. This family includes:

- **AltiCoreSWP (software platform)** — delivered for general compute on laptops, desktops, servers supporting Windows, Linux, CUDA and potentially macOS.
- **AltiCoreHDL** — FPGA/ASIC hardware inference and training.
- **AltiCoreMCU** — a toolset using the model training tools in AltiCoreSWP to deploy software inference to MCU-class devices, with support for **on-device (MCU) training** on compatible hardware.

Importantly, **AltiCoreSWP and AltiCoreMCU** support **both inference and training within the same framework**, meaning that — when memory and compute headroom permit — devices running AltiCore models can also **train or update them locally**.

At its current stage of development, **AltiCoreSWP's strongest demonstrated value** is as a **CPU-first inference engine** optimized for **compact, low-variance models**. In a CPU-only benchmark spanning **seven diverse public datasets**, AltiCoreSWP achieved **an order-of-magnitude higher inference throughput** than the fastest optimized neural-network CPU stack evaluated.

Note: Support for **very large parameter training and execution remains under active development**. EvoChip is not yet able to make formal commitments in this area.

SECTION 04

A Different Path to Faster Inference

This benchmark was run to answer a straightforward business question: **how fast can we run AI decisions on CPUs — and what does that unlock in real deployments?**

We compared CPU inference throughput between a standard **multilayer perceptron (MLP) neural network** and **AltiCoreSWP (Software Framework)**, a fundamentally different approach to model training and inference. By focusing on steady-state throughput rather than micro-latency, this study provides actionable insight for system architects, platform engineers, and decision-makers.

7	13–21×	17–28×	0
PUBLIC DATASETS	FASTER — WORKSTATION	FASTER — SERVER	DATASETS ALTICORE SLOWER

AltiCoreSWP delivered **order-of-magnitude higher throughput — between 13 and 21 times faster on a workstation CPU** and **between 17 and 41 times faster on a server CPU** — with **no dataset where AltiCoreSWP was slower**. The results demonstrate a **substantial and systematic throughput advantage**, highlighting the potential of non-neural, mathematically distinct inference architectures.

SECTION 05

Goal and Intent of the Benchmark

This benchmark informs a practical business decision: **is AltiCoreAI meaningfully faster for CPU inference than a conventional neural network approach, and is that advantage consistent enough to plan real deployments around it?** We were not trying to win a leaderboard — we were trying to establish whether AltiCoreAI's core promise shows up reliably under fair, repeatable measurement. Four specific questions guided the work:

- **Is the throughput advantage real against strong baselines?** Tested against mature, optimized CPU inference approaches including C++ TensorFlow Lite with XNNPACK.
- **Is the advantage consistent across a varied set of real tasks?** Seven public datasets reduce the risk of results driven by a single dataset's quirks.
- **Is the result robust with respect to the neural network method of execution?** The same MLP model was measured through four different CPU inference implementations.
- **Does the advantage hold across different CPU classes?** Measured on both a workstation CPU and a server CPU.

This report is about baseline CPU inference throughput. It does not characterize GPU-accelerated performance, end-to-end application latency, or very large-parameter training regimes still under development.

SECTION 06

Methodology

6.1 — OBJECTIVE AND SCOPE

This benchmark quantifies **maximum sustained CPU inference throughput** in **inferences per second**. Each input row = one inference. **Inference latency is explicitly out of scope.**

6.2 — SYSTEMS UNDER TEST

SYSTEM	CPU	RAM	ENVIRONMENT
Dell Precision 8560 (Workstation)	Intel Core i7-13700H	32 GB	`Windows 10 Pro`
Dell Server	Intel Xeon Gold 5416S	64 GB	`Windows 10 VM on ESXi`

All evaluated methods used **AVX2-capable CPU execution paths**. The server CPU supported AVX512 but this instruction set was not used.

6.3 — EXPERIMENTAL DESIGN

Each configuration is defined by hardware platform, dataset, and inference method. For each unique (hardware, dataset, method) configuration: **five (5) independent replicates**.

6.4 — DEFINITION OF A RUN AND REPLICATE PROTOCOL

Each run = fresh process launch executing: (1) load input data — outside timed region; (2) load model artifact — outside timed region; (3) allocate output buffers — outside timed region; (4) warm-up — outside timed region; (5) **timed inference region**; (6) record throughput metrics. Repeated five times per configuration.

6.5 — INFERENCE METHODS EVALUATED

- **out_00_genericPython** — Python Keras/TensorFlow baseline, default behavior, no tuning.
- **out_01_MTpython** — Python Keras/TensorFlow with explicit multithreading and empirically tuned batch sizes.
- **out_02_cppTFMT_std_XNN** — C++ TensorFlow Lite with XNNPACK; parallelism managed internally.
- **out_03_cppTFMT_cust_RUY** — C++ TensorFlow Lite with application-managed parallelism, custom RUY-based build.
- **out_04_AlticoreMT** — Multithreaded AltiCore (EvoChip) inference, mathematically distinct model family optimized for high-throughput CPU execution.

6.6 — TIMING INSTRUMENTATION

AltiCoreSWP (C++): `std::chrono::high_resolution_clock` — C++ TensorFlow Lite: `std::chrono::steady_clock` — Python: `time.perf_counter()`.

6.7 — TIMED REGION AND CONTROLS

Start: immediately before inference. Stop: immediately after inference. All warm-up outside timed region. No file I/O in timed region. Performance constrained by **CPU compute and memory bandwidth**.

6.8 — THROUGHPUT COMPUTATION

Throughput (inferences/sec) = N / t — N = rows processed, t = elapsed time in seconds.

6.9 — ROW COUNT AND MINIMUM DURATION

Timed region duration ≥ 0.10 seconds (100 ms). Row counts selected empirically. Batch size tuned for TensorFlow-based methods.

6.10 — PARALLELISM CONFIGURATION (PYTHON MULTITHREADED)

ENVIRONMENT VARIABLE	VALUE
CUDA_VISIBLE_DEVICES	-1
TF_ENABLE_ONEDNN_OPTS	1
TF_NUM_INTRAOP_THREADS	PHYS
TF_NUM_INTEROP_THREADS	PHYS
OMP_NUM_THREADS	PHYS
KMP_BLOCKTIME	1
KMP_AFFINITY	granularity=fine,scatter,1,0

6.11 — COMPILATION SETTINGS (C++)

/O2 — maximize speed optimizations. /arch:AVX2 — enable AVX2 instruction generation.

6.12 & 6.13 — EXECUTION CONDITIONS AND DATA COLLECTION

All configurations executed as five independent process launches under consistent system conditions. Consolidated dataset includes hardware, dataset, method identifiers, throughput, inference time, and rows processed. The consolidated CSV file is the **canonical data source** for all analysis and reporting.

SECTION 07

Model Overview: Quality, Footprint, and Inputs

7.1 — MODEL TRAINING AND SELECTION

Neural Network (MLP): TensorFlow + Keras, best model selected. **AltiCoreSWP:** automatic mode, best model selected. **Equal effort:** ~20 minutes each, full variable access, no preprocessing.

7.2 — INPUTS AND VARIABLE USAGE

AltiCoreSWP naturally produces models using a **minimal feature set**. MLP typically uses **all provided input variables**. Reduced inputs lower integration overhead and enable constrained-environment deployments.

7.3 — MODEL QUALITY AND FOOTPRINT

UCI-HAR excluded from this report.

DATASET	MCC ALTICOREAI / NN	ACCURACY ALTICOREAI / NN	PARAMETERS ALTICOREAI / NN	ARITH. OPS ALTICORESWP / NN	VARIABLES ALTICORESWP / NN
Credit Default UCI	0.412 / 0.499	0.802 / 0.816	**66 / 4,281**	116 / 8,323	8 / 23

DATASET	MCC ALTICOREAI / NN	ACCURACY ALTICOREAI / NN	PARAMETERS ALTICOREAI / NN	ARITH. OPS ALTICORESWP / NN	VARIABLES ALTICORESWP / NN
Credit Final	0.808 / 0.825	0.999 / 0.999	**36 / 1,272**	62 / 2,463	6 / 31
Give Me Some Credit	0.380 / 0.336	0.913 / 0.935	**62 / 10,751**	110 / 21,103	10 / 10
Intelligent Mfg. High Eff.	1.000 / 0.968	1.000 / 0.998	**66 / 10,017**	116 / 19,619	7 / 9
Intelligent Mfg. Low Eff.	1.000 / 0.992	1.000 / 0.977	**66 / 10,017**	116 / 19,619	5 / 9
Machine Failure	0.382 / 0.462	0.985 / 0.988	**62 / 3,381**	110 / 6,523	5 / 5
SPECT	0.628 / 0.517	0.866 / 0.836	**36 / 10,849**	62 / 21,283	7 / 22

SECTION 08

What These Characteristics Imply for Inference Throughput and Deployment

8.1 — COMPUTE FOOTPRINT DRIVES THROUGHPUT HEADROOM

Across seven datasets, selected AltiCoreSWP models are dramatically smaller: **parameters: 35× to 301× fewer** — **arithmetic operations per inference: ~40× to 343× fewer**. These structural differences reduce work per inference and explain why large throughput differences are achievable in steady-state CPU execution.

8.2 — INPUT REQUIREMENTS AFFECT INTEGRATION AND DEPLOYMENT REACH

AltiCoreAI uses fewer variables than the MLP model in most datasets (and never more): fewer upstream data feeds, fewer fragile dependencies, better fit for constrained environments where only a subset of signals is available.

8.3 — QUALITY CONTEXT FOR INTERPRETING SPEED

AltiCoreAI matches or exceeds the MLP on both MCC and accuracy in **3 datasets**, trails in **3 datasets**, and shows a mixed profile in **1 dataset**.

SECTION 09

CPU Inference Throughput Results

Throughput reported as **inferences per second**. Five independent process launches per configuration; results use **median throughput**.

9.1 — HEADLINE RESULT

~13×	~17×	27.6×	575 M
FASTER — WORKSTATION (TYPICAL)	FASTER — SERVER (TYPICAL)	PEAK SPEEDUP — SERVER	PEAK INF/SEC — SERVER

On the **workstation-class CPU**, AltiCoreSWP was typically **~13× faster** (range ~6.7× to ~21×). On the **server-class CPU**, typically **~17× faster** (range ~9.1× to ~27.6×). In **every dataset**, AltiCoreSWP delivered higher throughput than the best-performing neural-network implementation.

In absolute terms: **~225–361 M inferences/sec** on the workstation and **~472–575 M inferences/sec** on the server. The fastest NN baseline (C++ TFLite XNNPACK) sustained ~17–48 M inferences/sec on workstation and ~21–54 M on server.

9.2 — WHAT "FASTEST NN CPU IMPLEMENTATION" MEANS

In every dataset on both platforms, the fastest neural-network configuration was **C++ TensorFlow Lite with XNNPACK** — a mature, optimized, production-class CPU inference path.

9.3 — SPEEDUP BY DATASET

DATASET	SPEEDUP — WORKSTATION	SPEEDUP — SERVER
Credit Default UCI	6.90×	15.70×
Credit Fraud	7.50×	17.20×
Give Me Some Credit	13.70×	14.90×
Intelligent Manufacturing High Efficiency	12.90×	18.60×
Intelligent Manufacturing Low Efficiency	13.00×	19.00×
Machine Failure	6.70×	9.10×
SPECT	21.00×	27.60×

9.4 — WHY THESE RESULTS MATTER IN REAL DEPLOYMENTS

- **Capacity per server:** More decisions per machine without adding hardware.
- **Cost per decision:** Higher throughput reduces compute required per unit of work.
- **Broader CPU-first deployment:** More AI workloads remain viable without GPUs.

9.5.1 — WORKSTATION CPU: DELL PRECISION 8560 (WINDOWS 10 PRO)

Median inferences/sec — 5 runs. All values in millions.

DATASET	PYTHON BASELINE	PYTHON THREADED	C++ TFLITE XNN	C++ TFLITE RUY	ALTICORE MT (AVX2)
Credit Default	0.059	5.55	33.47	8.37	225.31
Credit Fraud	0.072	7.14	48.18	17.74	361.01
Give Me Some Credit	0.070	6.42	19.79	6.27	271.37

DATASET	PYTHON BASELINE	PYTHON THREADED	C++ TFLITE XNN	C++ TFLITE RUY	ALTICORE MT (AVX2)
Intelligent Mfg. High Eff.	0.045	5.08	19.60	3.97	**252.11**
Intelligent Mfg. Low Eff.	0.043	5.07	20.06	3.58	**260.40**
Machine Failure	0.066	8.21	40.52	10.90	**270.98**
SPECT	0.044	4.36	17.07	3.89	**358.32**
UCI_HAR	0.058	1.11	1.89	3.72	**341.19**

9.5.2 — SERVER CPU: XEON GOLD 5416S (WINDOWS 10 VM ON ESXI)

Median inferences/sec — 5 runs. All values in millions.

DATASET	PYTHON BASELINE	PYTHON THREADED	C++ TFLITE XNN	C++ TFLITE RUY	ALTICORE MT (AVX2)
Credit Default UCI	0.042	4.82	30.11	11.76	**472.20**
Credit Fraud	0.043	6.06	30.96	30.09	**531.51**
Give Me Some Credit	0.041	7.77	32.94	9.04	**492.28**
Intelligent Mfg. High Eff.	0.026	4.64	25.54	5.26	**475.91**
Intelligent Mfg. Low Eff.	0.026	4.59	25.81	5.06	**491.17**
Machine Failure	0.042	5.79	53.89	15.19	**492.59**
SPECT	0.026	4.01	20.83	5.23	**574.84**
UCI_HAR	0.035	1.09	2.57	7.88	**499.92**

SECTION 10

What This Unlocks for Business

AltiCore's CPU throughput advantage is a direct lever on **unit economics** and **deployment reach** — the two realities that determine whether AI scales in the real world.

10.1 — WHAT THIS MEANS FOR COST AND CAPACITY

- **Higher capacity per server.** A 10–50× throughput uplift translates into dramatically more decisions per machine — deferring hardware expansion and reducing fleet size.
- **Lower cost per decision.** Higher sustained inferences/sec reduces infrastructure needed to deliver the same outcomes.

- **Less engineering spent "making the baseline run fast."** AltiCore's advantage held against a highly optimized NN stack across four different NN implementations.

> **EXECUTIVE TRANSLATION**
>
> - If inference throughput is limiting growth or driving expenditures, AltiCore creates immediate room to scale without a proportional increase in infrastructure.

10.2 — WHAT THIS MEANS FOR DEPLOYMENT REACH

- **CPU-first production systems.** AltiCore's demonstrated throughput advantage makes CPU-first deployments more viable for larger workloads and tighter SLAs.
- **Constrained environments where "nothing else can run."** Model compactness and reduced input requirements enable edge and embedded deployments. The critical question is often not "is it faster?" but **"can it run at all?"**
- **Data-limited surfaces.** AltiCore's automatic discovery of a minimal variable set makes deployment feasible where maintaining all signals is impractical.

10.3 — WHAT THIS MEANS FOR PRODUCT STRATEGY

1. **Replace inference where the model is compact and call volume is high.** Target always-on, high-frequency, CPU-bound decision points: risk screening, fraud gating, operational classification, industrial monitoring.
2. **Use AltiCore as a "frontline" decision layer.** A compact, high-throughput model acts as a gating layer — decide when to escalate to more expensive compute and when a fast local decision is sufficient.
3. **Deploy intelligence closer to the source.** Where connectivity, latency, cost, or privacy constraints make centralized inference unattractive, AltiCore's compact-model orientation — paired with MCU and HDL delivery options — supports distributing inference into edge and embedded environments.

Because **AltiCoreAI supports both inference and training within the same framework**, it enables a practical pattern: **train or update where you deploy** when memory and compute headroom permit.

SECTION 11

Conclusion

The benchmark results demonstrate that **AltiCoreSWP delivers a sustained, order-of-magnitude throughput advantage** over highly optimized neural-network inference stacks under CPU, steady-state execution conditions across diverse structured datasets.

These findings challenge the prevailing assumption that neural networks are the default optimal solution for all inference workloads. For high-volume, structured decisioning tasks where **cost efficiency, predictability, and deployment simplicity** dominate, **AltiCoreAI's** mathematically distinct approach offers a compelling alternative.

The implications extend beyond software: the same efficiency characteristics that enable superior CPU performance also align naturally with **edge, embedded, and hardware-integrated deployments**, reinforcing the architectural coherence of the AltiCoreAI platform across AltiCoreSWP, AltiCoreMCU, and AltiCoreHDL.

While large-scale training and very high-parameter models remain under active development, the current results establish **AltiCoreAI** as a **highly competitive inference engine** for a broad and economically significant class of

real-world workloads. Higher throughput creates immediate headroom to serve more decisions on the same CPU footprint or reduce the footprint required for current volumes.

SECTION 12

Datasets

- **Credit card fraud detection** — kaggle.com/code/myusage16/credit-card-fraud-with-neural-network-100-accurcy
- **Machine Failure** — kaggle.com/code/ashx010/binary-machine-failure-prediction-model
- **Credit Final** — kaggle.com/code/ashx010/binary-machine-failure-prediction-model
- **Give Me Some Credit** — github.com/ectormgl/givemecredit
- **Machine Failure Low and High Efficiency** — kaggle.com/code/satyaprakashshukl/nn-machine-failure-series-3-e-17

The following section (13) presents detailed quantitative results, dataset-specific analysis, and scaling behavior across platforms.

SECTION 13

Detailed Quantitative Results

13.1 — ALTICOREAI VS. NEURAL NETWORKS — DELL PRECISION 8560 LAPTOP

Median across runs. All values in millions of inferences/sec.

DATASET	PYTHON BASELINE	PYTHON THREADED	C++ TFLITE XNN	C++ TFLITE RUY	ALTICORE MT (AVX2)
Credit Default	0.059	5.55	33.47	8.37	**225.31**
Credit Fraud	0.072	7.14	48.18	17.74	**361.01**
Give Me Some Credit	0.070	6.42	19.79	6.27	**271.37**
Intelligent Mfg. High Eff.	0.045	5.08	19.60	3.97	**252.11**
Intelligent Mfg. Low Eff.	0.043	5.07	20.06	3.58	**260.40**
Machine Failure	0.066	8.21	40.52	10.90	**270.98**
SPECT	0.044	4.36	17.07	3.89	**358.32**

Speedup vs. each NN method — in multiples.

DATASET	VS. OUT-OF-BOX NN	VS. MULTICORE PYTHON	VS. C++ TFLITE XNN	VS. C++ TFLITE RUY
Credit Default	**3,798×**	41×	7×	27×

DATASET	VS. OUT-OF-BOX NN	VS. MULTICORE PYTHON	VS. C++ TFLITE XNN	VS. C++ TFLITE RUY
Credit Fraud	**5,011×**	51×	7×	20×
Give Me Some Credit	**3,879×**	42×	14×	43×
Mfg. High Eff.	**5,664×**	50×	13×	64×
Mfg. Low Eff.	**6,012×**	50×	13×	73×
Machine Failure	**3,921×**	32×	10×	24×
SPECT	**8,165×**	82×	21×	92×

13.2 — ALTICOREAI VS. NEURAL NETWORKS — DELL SERVER XEON GOLD 5416S

All values in millions of inferences/sec.

DATASET	PYTHON BASELINE	PYTHON THREADED	C++ TFLITE XNN	C++ TFLITE RUY	ALTICORE MT (AVX2)
Credit Default UCI	0.042	4.82	30.11	11.76	**472.20**
Credit Fraud	0.043	6.06	30.96	30.09	**531.51**
Give Me Some Credit	0.041	7.77	32.94	9.04	**492.28**
Intelligent Mfg. High Eff.	0.026	4.64	25.54	5.26	**475.91**
Intelligent Mfg. Low Eff.	0.026	4.59	25.81	5.06	**491.17**
Machine Failure	0.042	5.79	53.89	15.19	**492.59**
SPECT	0.026	4.01	20.83	5.23	**574.84**

Speedup vs. each NN method — in multiples.

DATASET	VS. OUT-OF-BOX NN	VS. MULTICORE PYTHON	VS. C++ TFLITE XNN	VS. C++ TFLITE RUY
Credit Default	**11,206×**	98×	40×	40×
Credit Fraud	**12,488×**	88×	17×	18×
Give Me Some Credit	**11,882×**	63×	15×	54×
Mfg. High Eff.	**18,482×**	103×	19×	90×
Mfg. Low Eff.	**19,119×**	107×	19×	97×
Machine Failure	**11,801×**	85×	9×	32×
SPECT	**22,376×**	143×	28×	110×

13.3 — ALTICOREAI (DELL 8560 LAPTOP) VS. NEURAL NETWORKS (XEON GOLD 5416S SERVER)

Inference gains — AltiCore on laptop vs. best NN on server.

DATASET	ALTICOREAI GAIN
SPECT	17.2×
Credit Final	11.7×
Intelligent Manufacturing Low Efficiency	10.1×
Intelligent Manufacturing High Efficiency	9.9×
Give Me Some Credit	8.2×
Credit Default UCI	7.5×
Machine Failure	5.0×

13.4 — MCC (MATTHEWS CORRELATION COEFFICIENT)

DATASET	ALTICOREAI	NEURAL NETWORKS
Credit Default UCI	0.411929	0.499432
Credit Final	0.807504	0.824992
Give Me Some Credit	0.379689	0.336282
Intelligent Manufacturing High Efficiency	1.000000	0.967743
Intelligent Manufacturing Low Efficiency	0.999884	0.992433
Machine Failure	0.381629	0.461761
SPECT	0.628332	0.516587

13.5 — ACCURACY

DATASET	ALTICOREAI	NEURAL NETWORKS
Credit Default UCI	0.802083	0.816250
Credit Final	0.999320	0.999407
Give Me Some Credit	0.912533	0.934667
Intelligent Manufacturing High Efficiency	1.000000	0.998120
Intelligent Manufacturing Low Efficiency	0.999960	0.977400
Machine Failure	0.984699	0.987649
SPECT	0.865672	0.835821

13.6 — PARAMETER COUNT

DATASET	ALTICOREAI	NEURAL NETWORKS
Credit Default UCI	66	4,281
Credit Final	36	1,272
Give Me Some Credit	62	10,751
Intelligent Manufacturing High Efficiency	66	10,017
Intelligent Manufacturing Low Efficiency	66	10,017
Machine Failure	62	3,381
SPECT	36	10,849

13.7 — ARITHMETIC OPERATIONS PER INFERENCE

DATASET	ALTICOREAI	NEURAL NETWORKS
Credit Default UCI	116	8,323
Credit Final	62	2,463
Give Me Some Credit	110	21,103
Intelligent Manufacturing High Efficiency	116	19,619
Intelligent Manufacturing Low Efficiency	116	19,619
Machine Failure	110	6,523
SPECT	62	21,283